DATA ENGINEERING
ROADMAP 2025

A complete, structured guide from absolute beginner to production engineer

6 Phases • 50+ Hours of Practice • 20 Real-World Problems

6	4	20+	~60
Phases	Months Est.	Problems	Technologies

HOW TO USE THIS ROADMAP

This roadmap is designed to take you from zero to a job-ready data engineer in approximately 4-6 months of dedicated study (~4-6 hours/day). Each phase builds on the previous. **Do not skip phases** — the problems progressively depend on earlier skills.

Step 1	Read the topics for the phase and gather resources (see recommended tools per phase).
Step 2	Build small projects for each topic before moving to phase problems.
Step 3	Solve ALL problems in a phase before advancing. Don't just read — build.
Step 4	Push every project to GitHub with a proper README and architecture diagram.
Step 5	After Phase 4+, start applying to jobs — you already have the core skills.

DIFFICULTY LEVELS

Beginner	Can be done in hours. Focus on syntax & basic usage.
Intermediate	Requires research. Expect 1-3 days of work.
Advanced	Multi-day projects. Combine multiple tools.
Expert	Portfolio-level. May take 1-2 weeks.

1 PHASE 1 — Foundations

Programming, SQL & Core CS Concepts | ~8-12 weeks

Goal

- Build the bedrock. Without strong Python and SQL skills, everything else falls apart.

CORE TOPICS

Python for Data

- Variables, types, loops, functions
- List comprehensions, generators
- File I/O (CSV, JSON, Parquet)
- pandas, NumPy basics
- Virtual environments, pip, poetry

SQL Mastery

- SELECT, JOIN, GROUP BY, HAVING
- Window functions (ROW_NUMBER, RANK, LAG/LEAD)
- CTEs and subqueries
- Indexes and query execution plans
- PostgreSQL / MySQL hands-on

Linux & Command Line

- Bash scripting basics
- cron jobs for scheduling
- grep, awk, sed for log processing
- SSH, file permissions

Version Control

- Git: commit, branch, merge, rebase
- GitHub workflows / pull requests
- Writing good commit messages

TECHNOLOGIES TO LEARN

Python | pandas | SQL | PostgreSQL | Linux | Git | Bash

PRACTICE PROBLEMS

#01 Analyze a CSV Dataset

Beginne

Download the NYC Taxi dataset. Use pandas to clean it, remove nulls, fix data types, and compute average fare per borough per hour.

Hint: Use pd.read_csv() with dtype hints

Hint: groupby + agg for aggregations

#02 SQL Window Functions Challenge

Beginne

Given an orders table (order_id, customer_id, amount, date), write a query to find the top 3 orders per customer using window functions.

Hint: ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY amount DESC)

#03 ETL with Pure Python

Interm

Write a Python script that reads a JSON API response, flattens nested fields, validates data types, and writes clean rows to a CSV.

Hint: json.loads(), csv.DictWriter

Hint: Use dataclasses for schema validation

#04 Log Parser CLI Tool

Interm

Build a CLI tool that reads Nginx access logs, counts unique IPs, identifies top endpoints, and outputs a summary report.

Hint: argparse for CLI args

Hint: collections.Counter for frequency

2 PHASE 2 — Data Pipelines & ETL

Orchestration, Ingestion & Transformation | ~10-14 weeks

Goal

- Learn to move and transform data reliably at scale. This is the heart of data engineering.

CORE TOPICS

ETL vs ELT Patterns

- Extract: APIs, databases, files, streams
- Transform: cleaning, enriching, aggregating
- Load: data warehouses, data lakes
- Idempotency & reprocessing strategies

Apache Airflow

- DAGs, Tasks, Operators
- TaskFlow API (Python decorators)
- Connections & Variables
- XComs for task communication
- Retries, SLAs, alerting

dbt (Data Build Tool)

- Models, Sources, Seeds, Snapshots
- Jinja templating in SQL
- Incremental models
- Tests: schema tests, custom tests
- Lineage graphs & documentation

Data Quality

- Great Expectations / Soda
- Schema validation
- Anomaly detection basics
- Data contracts concept

TECHNOLOGIES TO LEARN

Airflow | dbt | REST APIs | Great Expectations | Docker | Make

PRACTICE PROBLEMS

#01 Weather Pipeline

Beginne

Build an Airflow DAG that fetches weather data from OpenWeatherMap API every 6 hours, stores raw JSON in a local folder, and loads parsed data into PostgreSQL.

Hint: Use SimpleHttpOperator or PythonOperator

Hint: Store raw + processed separately

#02 dbt Transformation Project

Interm

Given raw e-commerce tables (orders, customers, products), build a dbt project with staging models, a fact_orders model, and dim_customers. Add tests for uniqueness and not_null.

Hint: sources.yml for raw tables

Hint: ref() function for dependencies

#03 Backfill Idempotent Pipeline

Interm

Design an Airflow DAG with catchup=True that can safely re-run for any historical date without duplicating data. Use an upsert strategy.

Hint: INSERT ON CONFLICT DO UPDATE in Postgres

Hint: Partition data by execution_date

#04 Data Quality Checks

Advance

Add Great Expectations checkpoints to a pipeline that validates: row counts didn't drop >10%, no nulls in primary keys, revenue column is always positive.

Hint: ExpectationSuite + Checkpoint API

Hint: Fail pipeline on validation failure

3 PHASE 3 — Data Warehousing & Storage

Cloud DWH, Data Lakes & Storage Formats | ~8-10 weeks

Goal

- Understand where and how data lives at scale. Storage choices affect cost and query speed dramatically.

CORE TOPICS

Cloud Data Warehouses

- BigQuery / Snowflake / Redshift architecture
- Columnar storage & compression
- Partitioning & clustering strategies
- Cost optimization techniques
- Materialized views

Storage Formats

- Parquet: columnar, compression, schema evolution
- Avro: row-based, Kafka favorite
- ORC: Hive-optimized
- Delta Lake / Iceberg: ACID on data lakes
- When to use each format

Data Lake Architecture

- Bronze / Silver / Gold medallion pattern
- S3 / GCS / Azure Blob storage
- Data catalog basics (Glue, Dataplex)
- Hive metastore concepts

Dimensional Modeling

- Star schema vs Snowflake schema
- Fact tables & dimension tables
- Slowly Changing Dimensions (SCD Types 1, 2, 3)
- Kimball methodology basics

TECHNOLOGIES TO LEARN

BigQuery | Snowflake | Parquet | Delta Lake | S3/GCS | dbt Advanced

PRACTICE PROBLEMS

#01 Design a Star Schema

Interm

For a retail company with orders, products, customers, and stores, design a star schema. Create fact_sales, dim_customer, dim_product, dim_date, dim_store. Justify your grain choice.

Hint: Grain = one row per order line item

Hint: Date dimension should be pre-built

#02 Parquet Optimization

Interm

Take a 5GB CSV file. Convert it to Parquet with optimal partitioning by year/month. Compare query times in DuckDB for a date-range filter query before and after.

Hint: Use pyarrow or pandas to_parquet()

Hint: Measure with %timeit or time module

#03 SCD Type 2 in dbt

Advance

Model a customer address table as SCD Type 2 using dbt snapshots. When a customer updates their city, the old record should be closed with an end_date and a new record opened.

Hint: Use dbt snapshot with strategy: timestamp

Hint: check_cols: ['city', 'state']

#04 BigQuery Cost Optimization

Advance

Given a 1TB table queried daily, implement partitioning by date and clustering by customer_id. Write 3 sample queries and explain how the partition pruning reduces bytes scanned.

Hint: PARTITION BY DATE(_PARTITIONTIME)

Hint: CLUSTER BY reduces sort scan

4 PHASE 4 — Distributed Processing

Spark, Flink & Large-Scale Computation | ~10-12 weeks

Goal

- When data exceeds memory, you need distributed computing. Spark is the industry standard.

CORE TOPICS

Apache Spark

- ■ RDDs vs DataFrames vs Datasets
- ■ Transformations (lazy) vs Actions (eager)
- ■ Spark SQL & DataFrame API
- ■ Partitioning, shuffling, spills
- ■ PySpark on local + cluster mode
- ■ Catalyst optimizer basics

Spark Streaming

- ■ Structured Streaming basics
- ■ Watermarks & late data handling
- ■ Triggers: micro-batch vs continuous
- ■ stateful operations

Performance Tuning

- ■ Broadcast joins for small tables
- ■ Repartition vs Coalesce
- ■ Caching & persistence levels
- ■ Monitoring with Spark UI

Stream Processing with Kafka

- ■ Kafka producers & consumers
- ■ Topics, partitions, consumer groups
- ■ Kafka Connect for connectors
- ■ Schema Registry & Avro

TECHNOLOGIES TO LEARN

PySpark | Kafka | Databricks | Spark SQL | Flink basics | YARN/K8s

PRACTICE PROBLEMS

#01 Word Count to Join

Beginne

Start with Spark word count. Then progress: join two large DataFrames (users + events) efficiently. Identify if a broadcast join is possible and implement it.

Hint: sc.textFile() for RDD version

Hint: df.hint('broadcast') for small tables

#02 Log Processing at Scale

Interm

Process 50M web server log lines with PySpark. Extract status codes, parse timestamps, compute p50/p95/p99 response times per endpoint per day.

Hint: Use regexp_extract for parsing

Hint: percentile_approx() for percentiles

#03 Streaming Kafka Pipeline

Advance

Set up a local Kafka cluster. Produce clickstream events. Write a Spark Structured Streaming job that aggregates clicks per product per minute and writes to PostgreSQL.

Hint: Use kafka-python or confluent-kafka

Hint: foreachBatch() sink for Postgres writes

#04 Delta Lake ACID Pipeline

Advance

Build a medallion pipeline on Delta Lake: raw JSON -> Bronze (append-only) -> Silver (deduped, typed) -> Gold (aggregated). Use MERGE for upserts in Silver.

Hint: delta.tables.DeltaTable.forPath()

Hint: MERGE INTO with matched/not matched

5 PHASE 5 — Cloud & DataOps

Cloud Platforms, CI/CD & Production Engineering | ~8-10 weeks

Goal

- Production data engineering means reliable, observable, automated infrastructure.

CORE TOPICS

Cloud Platforms

- AWS: S3, Glue, EMR, Redshift, Lambda, Step Functions
- GCP: BigQuery, Dataflow, Pub/Sub, Cloud Composer
- Azure: ADLS, Synapse, Data Factory (overview)
- Serverless vs managed services tradeoffs

Infrastructure as Code

- Terraform for cloud resources
- Docker & Docker Compose
- Kubernetes basics (pods, deployments, services)
- Helm charts for Airflow/Spark

CI/CD for Data

- GitHub Actions for dbt + Airflow
- Automated testing in pipelines
- Blue/green deployments for pipelines
- Environment management (dev/staging/prod)

Observability

- DataDog / Grafana / Prometheus for metrics
- Structured logging with structlog
- Alerting on pipeline failures
- Data lineage tracking (OpenLineage, Marquez)

TECHNOLOGIES TO LEARN

AWS/GCP | Terraform | Docker | Kubernetes | GitHub Actions | Grafana

PRACTICE PROBLEMS

#01 Dockerize an ETL Pipeline

Interm

Package your Phase 2 Airflow pipeline in Docker Compose (Airflow + Postgres + your custom operator). Make it runnable with a single docker-compose up command.

Hint: Use official apache/airflow image

Hint: Mount DAGs folder as a volume

#02 Terraform a Data Stack

Advance

Write Terraform code to provision: an S3 bucket with versioning, a Redshift cluster, an IAM role for Glue, and a Glue crawler. Use modules and variables.

Hint: aws_s3_bucket, aws_redshift_cluster resources

Hint: Use terraform.tfvars for secrets

#03 CI/CD for dbt

Advance

Set up a GitHub Actions workflow that: runs dbt compile on PR, runs dbt test on merge to main, and posts test results as a PR comment.

Hint: Use dbt-action or custom python step

Hint: dbt source freshness + dbt test --select state:modified

#04 End-to-End Monitoring

Advance

Instrument your pipeline with structured logs. Build a Grafana dashboard showing: DAG success rate, task duration p95, data freshness lag, and row count anomaly alerts.

Hint: Use Airflow statsd metrics

Hint: Alertmanager for threshold alerts

6 PHASE 6 — Capstone Projects

Real-World Portfolio Projects | ~6-8 weeks

Goal

- Cement your skills by building production-grade end-to-end systems. These go on your portfolio/GitHub.

CORE TOPICS

Project Ideas

- Real-time analytics dashboard (Kafka + Spark + Grafana)
- dbt + BigQuery data warehouse for public dataset
- Lakehouse on AWS (S3 + Delta + Athena + dbt)
- ML Feature Store pipeline (Feast or custom)

What Makes a Strong Portfolio Project

- Has a real business problem, not just 'load CSV to DB'
- Includes data quality checks & tests
- Has a README with architecture diagram
- Handles edge cases: late data, schema changes, failures

TECHNOLOGIES TO LEARN

All of the above | System Design | Documentation | Communication

PRACTICE PROBLEMS

#01 Streaming E-Commerce Analytics

Advance

Build end-to-end: Kafka produces fake e-commerce events -> Spark Structured Streaming aggregates -> Delta Lake stores -> dbt transforms -> Grafana visualizes orders per minute, revenue, top products.

Hint: Use faker library for event generation

Hint: Grafana + Delta Lake via Trino connector

#02 Public Dataset Warehouse

Advance

Pick a large public dataset (NYC Taxi, GitHub Archive, or Stack Overflow). Design a full DWH with star schema in BigQuery/Snowflake, dbt models, data quality tests, and an analytics layer.

Hint: Ingest with Airbyte or custom scripts

Hint: Expose Gold layer via Looker Studio or Metabase

#03 Open Source Lakehouse

Expert

Build a complete Lakehouse: MinIO (local S3) + Apache Iceberg + Trino + dbt + Apache Superset. Automate with Airflow. Handle schema evolution and time travel queries.

Hint: Use docker-compose to orchestrate all services

Hint: Iceberg MERGE INTO for CDC patterns

#04 Real-time Anomaly Detection

Expert

Stream sensor/metric data through Kafka. Use Spark Streaming + a simple statistical model (z-score or IQR) to detect anomalies in real-time. Alert via Slack webhook when anomaly detected.

Hint: Rolling window statistics with watermarks

Hint: requests library for Slack webhook

RECOMMENDED RESOURCES

Books

- Fundamentals of Data Engineering — Joe Reis & Matt Housley
- Designing Data-Intensive Applications — Martin Kleppmann
- The Data Warehouse Toolkit — Ralph Kimball
- Learning Spark (2nd ed) — Damji, Wenig, Das, Lee

Free Online Courses

- DataTalks.Club Data Engineering Zoomcamp (GitHub — free)
- dbt Learn — learn.getdbt.com (free)
- Apache Airflow Astronomer Academy (free tier)
- Google BigQuery for Data Engineers — Coursera

Practice Datasets

- NYC Taxi & Limousine Commission Data (public)
- GitHub Archive — gharchive.org
- Stack Overflow Annual Survey Data
- Kaggle Datasets: eCommerce, Retail, Finance

Communities

- DataTalks.Club Slack — active DE community
- dbt Community Slack — #general, #advice-for-learners
- r/dataengineering on Reddit
- LinkedIn — follow Seattle Data Guy, Zach Wilson, Joe Reis

Good luck on your Data Engineering journey! Remember: build things, break things, and fix them. The problems in this roadmap are designed to mirror real interview questions and on-the-job tasks.